

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет
по Домашней работе №3
«Тестирование API средствами Postman»

Выполнил:
Суворин Иван
БР 2.2

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задание

- Реализовать тестирование API средствами Postman;
- Написать тесты внутри Postman.

Требовалось получить один комплексный сценарий по основному процессу приложения

Ход работы

1. Выбор сценария

Основной процесс сервиса обмена рецептами — это полный жизненный цикл рецепта с участием обеих ролей: обычный пользователь создает и публикует рецепт, администратор его модерирует, после чего рецепт становится доступен всем и с ним можно взаимодействовать (лайк, комментарий, избранное). Такой сценарий дает более полную проверку, чем в ЛР1, где из-за ручного тестирования в Swagger UI автор и модератор совпадали в одном аккаунте — здесь это исправлено: используются две разные учетные записи (новый пользователь + администратор из `npm run seed`).

2. Структура коллекции

Коллекция «Recipe Service — комплексный сценарий» состоит из 15 запросов, выполняющихся строго по порядку (Collection Runner проходит их сверху вниз). Каждый следующий запрос использует данные, полученные из ответа предыдущего, через переменные коллекции (`userToken`, `adminToken`, `recipeId`, `commentId`).

№	Запрос	Назначение и проверки
1	POST /auth/register	Регистрация нового пользователя со случайными username/email
2	POST /auth/login	Вход этим пользователем, сохранение userToken
3	POST /recipes	Создание рецепта (статус draft), сохранение recipeId
4	POST /recipes/{id}/submit	Отправка рецепта на модерацию (draft -> pending)
5	POST /auth/login	Вход администратором (аккаунт из npm run seed), сохранение adminToken
6	PUT /admin/recipes/{id}/approve	Одобрение рецепта (pending -> published)
7	GET /recipes	Публичный список — проверка, что рецепт виден и опубликован
8	GET /recipes?difficulty=easy	Фильтрация списка по сложности

№	Запрос	Назначение и проверки
9	GET /recipes/{id}	Просмотр детальной страницы рецепта
10	POST /recipes/{id}/like	Лайк рецепта
11	POST /comments/recipe/{id}	Добавление комментария, сохранение commentId
12	GET /comments/recipe/{id}	Проверка, что комментарий виден в списке
13	POST /recipes/{id}/favorite	Добавление рецепта в избранное
14	GET /users/me/recipes	Личный кабинет — рецепт виден среди «своих»
15	GET /users/me/favorites	Личный кабинет — рецепт виден в «избранном»

3. Повторяемость запуска

Чтобы коллекцию можно было запускать многократно без ошибки «пользователь уже существует», в Pre-request Script первого запроса генерируются случайные username и email на основе текущего времени:

```
"const rand = Date.now();",
"pm.collectionVariables.set('testUsername', 'user' + rand);",
"pm.collectionVariables.set('testEmail', 'user' + rand +
'@example.com');",
"pm.collectionVariables.set('testPassword', 'Password123');"
```

4. Написанные тесты

В каждом запросе на вкладке Tests написаны проверки через pm.test / pm.expect, проверяющие корректность HTTP-статуса и содержимого ответа. Часть тестов также сохраняет данные для следующих запросов цепочки. Пример — тест запроса «3. Создание рецепта»:

```
"pm.test('Статус 200', () => pm.response.to.have.status(200));",
"",
"pm.test('Пользователь создан с ролью user', () => {",
"  const json = pm.response.json();",
"  pm.expect(json).to.have.property('id');",
"  pm.expect(json.username).to.eql(pm.collectionVariables.get('testUs",
ername')));",
"  pm.expect(json.role).to.eql('user');",
"});"
```

Еще один пример — проверка запроса «7. Публичный список рецептов», где нужно найти конкретный объект в массиве, а не просто проверить статус:

```
"pm.test('Наш рецепт присутствует в публичном списке', () => {",
"  const json = pm.response.json();",
"  const recipeId = Number(pm.collectionVariables.get('recipeId'));",
"  const found = json.items.find((r) => r.id === recipeId);",
"  pm.expect(found, 'рецепт не найден в",
"/recipes').to.not.be.undefined;",
"  pm.expect(found.status).to.eql('published');",
"});"
```

```
"});"
```

Всего по коллекции написано 28 проверок (pm.test), распределенных по 15 запросам — от одной (простая проверка статус-кода для 204-ответов у лайка/избранного) до двух-трех на запрос.

5. Переменные

— Переменные окружения (Recipe Service Local): baseUrl, adminEmail, adminPassword — постоянные настройки, не меняющиеся в ходе прогона;

— Переменные коллекции: testUsername, testEmail, testPassword, userToken, adminToken, recipeId, commentId — заполняются динамически по ходу выполнения сценария и передают данные между запросами.

Демонстрация запуска

Коллекция запущена через Collection Runner (Postman, desktop-приложение). Результат — все 15 запросов и все 28 тестов внутри них пройдены успешно:

RUN SUMMARY			1
>	POST	1. Регистрация пользователя	2 0
>	POST	2. Вход пользователем	2 0
>	POST	3. Создание рецепта	2 0
>	POST	4. Отправка рецепта на модерацию	2 0
>	POST	5. Вход администратором	2 0
>	PUT	6. Одобрение рецепта администратором	2 0
>	GET	7. Публичный список рецептов	2 0
>	GET	8. Фильтрация списка по сложности	2 0
>	GET	9. Просмотр детальной страницы рецепта	2 0
>	POST	10. Лайк рецепта	1 0
>	POST	11. Добавление комментария	2 0
>	GET	12. Список комментариев рецепта	2 0
>	POST	13. Добавление рецепта в избранное	1 0
>	GET	14. Личный кабинет: мои рецепты	2 0
>	GET	15. Личный кабинет: избранные рецепты	2 0

Рисунок 1 — Порядок выполнения запросов в Collection Runner

All 28 Passed 28 Failed 0 Skipped 0 Errors 0 Console log

Рисунок 2 — Итоговая сводка прогона: 28 из 28 тестов пройдено, 0 ошибок

Журнал сервера во время прогона

Лог сервера (recipe-service, npm run dev) за время выполнения коллекции подтверждает, что все запросы дошли до бэкенда и обработаны с ожидаемыми статус-кодами:

```
POST /api/auth/register 200 361.381 ms - 142
POST /api/auth/login 200 75.491 ms - 168
POST /api/recipes 200 29.599 ms - 832
POST /api/recipes/3/submit 200 23.657 ms - 834
POST /api/auth/login 200 64.741 ms - 169
PUT /api/admin/recipes/3/approve 200 13.398 ms - 836
GET /api/recipes 200 4.494 ms - 1896
GET /api/recipes?difficulty=easy 200 3.233 ms - 1896
GET /api/recipes/3 200 4.554 ms - 836
POST /api/recipes/3/like 204 8.235 ms - -
POST /api/comments/recipe/3 200 22.523 ms - 243
GET /api/comments/recipe/3 200 8.884 ms - 245
POST /api/recipes/3/favorite 204 30.169 ms - -
GET /api/users/me/recipes 200 5.499 ms - 838
GET /api/users/me/favorites 200 3.551 ms - 856
```

Видно, что все запросы, вплоть до вложенных операций над одним и тем же рецептом (id = 3), выполнены в правильном порядке и с корректными кодами ответа (200 — есть тело ответа, 204 — успех без тела, как и предусмотрено для лайка/избранного).

ВЫВОД

В рамках ДЗ3 реализован комплексный тестовый сценарий, охватывающий основной процесс приложения «Recipe Service»: полный жизненный цикл рецепта от создания обычным пользователем до публикации администратором и последующего социального взаимодействия (лайк, комментарий, избранное). Сценарий оформлен как коллекция Postman из 15 последовательных запросов с 28 автоматическими проверками (pm.test), передающих данные друг другу через переменные. Коллекция воспроизводима — при каждом запуске создается новый пользователь со случайными данными, поэтому повторный прогон не приводит к конфликтам. Прогон через Collection Runner подтвержден: 28 из 28 тестов пройдено, ошибок и пропущенных проверок нет. В результате работы были созданы следующие файлы:

- postman/Recipe-Service.postman_collection.json — коллекция запросов с тестами,
- postman/Recipe-Service.postman_environment.json — окружение (baseUrl и данные администратора).